HG-RAG: Hierarchy-Guided Retrieval-Augmented Generation for Structured Knowledge Graphs

Pranav Yadav
University of California, Merced
Merced, CA, USA
pyadav@ucmerced.edu

Abstract

Retrieval Augmented Generation (RAG) has proven to be a widely successful process at improving the quality of outputs from a Large Language Model (LLM) for wider context. However, RAG systems typically retrieve context from flat document stores, which struggles when queries require hierarchical or relational reasoning across structured knowledge. I present HG-RAG (Hierarchy-Guided RAG), a framework that performs graph-traversal over a hierarchical knowledge graph to deliver structured context to a language model. My retrieval pipeline resolves a named entity anchor from the query, then expands context upward through parent nodes, laterally through relational neighbors, and downward through child nodes when needed. I evaluate HG-RAG against a dense retrieval baseline across three world scales (18–800 nodes) with four query types: `local fact`, `hierarchical`, `neighborhood`, and `multi_hop`. Results show HG-RAG consistently outperforms the flat baseline on hierarchical, relational, and multi-hop reasoning tasks, while reducing hallucination and maintaining locality coherence.

https://github.com/Pranubot/HG-RAG

1 Introduction

LLMs can often provide incorrect information when presented with a query since their knowledge consists purely of pre-training data (Ji et al., 2023; Shuster et al., 2021). A common workaround is to provide LLMs with additional context upfront with the query. However, the effectiveness of this method starts to break down when the context is too large to be tacked on the side, requiring a smarter system. Retrieval Augmented Generation aims to reduce this context build-up by using a user's query to search for semantically similar data, which are added to the model's context. This avoids the whole context, including data irrelevant to the query, from being attached.

Existing RAG frameworks assume flat, unstructured retrieval. When knowledge is inherently hierarchical (geopolitical structures, taxonomies, tree structures), dumping a flat context window onto your LLM loses relational signal and increases risk of hallucination (Shuster et al., 2021). Structured knowledge demands structured retrieval; a system that understands not just what a node is, but where it sits in the hierarchy. I address this by extending the RAG framework with graph-traversal subroutines that collect context from a named entity anchor upward through parent nodes, laterally through relational neighbors, and downward through children. This produces a focused subgraph of contextually relevant nodes rather than an unorganized document dump, giving the LLM the relational scaffolding it needs to reason accurately across hierarchical and multi-hop queries (Multi-hop queries require chaining multiple relational steps across the graph to arrive at an answer).

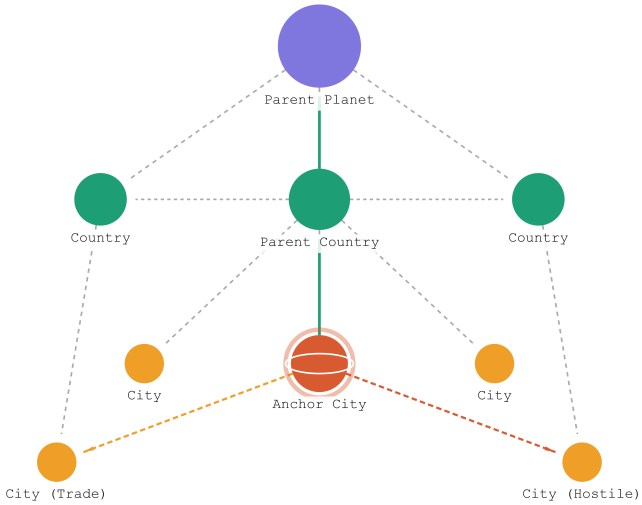

Figure 1: A small snapshot of what kind of graphs we are retrieving from and how HG-RAG visualizes them. We visualize relationships relative to our anchor node (red).

2 Related Work

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020) as a method for grounding LLM responses in external knowledge by retrieving relevant documents at inference time. While effective for open-domain question answering, standard RAG requires retrieved documents to be flat, unordered context.

Several works have explored extending RAG to graph-structured knowledge. Microsoft's GraphRAG (Edge et al., 2024) constructs a community-based knowledge graph from source documents and uses it to guide retrieval. This proved to be a fruitful development on the standard naive RAG framework. Similarly, knowledge graph question answering systems such as KGQA (Lan and Jiang, 2020) have shown that explicit graph traversal improves multi-hop reasoning by following relational edges. In a related direction, Yasunaga et al. (2021) jointly trains a graph neural network alongside a language model to reason over knowledge graph subgraphs. In contrast, dense retrieval methods (Karpukhin et al., 2020) rely on semantic similarity in a shared vector space, with no mechanism for exploiting relational structure. A broader survey of neural KGQA approaches (Chakraborty et al., 2021) further demonstrates that graph-structured retrieval consistently outperforms flat retrieval on relational reasoning tasks.

HG-RAG differs from these approaches in its explicit focus on *hierarchical* structure. Rather than treating all graph edges as equivalent, HG-RAG distinguishes between structural edges (`contains`) and relational edges (`borders`, `trade_with`, `hostile`), and uses a directional k-step traversal to collect context that respects the hierarchy. This makes HG-RAG exceptional for domains where knowledge is naturally organized into parent-child levels.

3 Setup

3.1 World Construction

To evaluate HG-RAG in a controlled setting, I construct a series of synthetic hierarchical knowledge graphs using a three-tier structure: Planets → Countries → Cities. These worlds are implemented as directed graphs using `NetworkX` (Hagberg et al., 2008). This hierarchy mirrors real-world geopolitical relations and provides a proper testbed for hierarchical reasoning. The framing is inspired by grand strategy video games where dense relational data between territories on multiple levels is the norm, making it a strong stress test for structured retrieval.

Each node carries domain-relevant attributes that the LLM will be assessed on. Planets store population, stability, and exports; cities store imports. Edges include both structural and relational information of six types: `contains` (children), `borders`, `trade_with`, `hostile`, `unfriendly`, and `neutral`. To guarantee valid ground-truth answers for multi-hop queries, city imports are post-assigned from their trade partners' exports after world generation. This is a semantic coupling heuristic that ensures every trade-chain query has a deterministic, resolvable answer grounded in the graph.

Size	Planets	Countries	Cities	Total Nodes
Small	2	3	3	∼18
Medium	3	5	10	∼150
Large	4	8	25	∼800

Table 1: World graph sizes used in testing.

This range allows us to observe how both retrieval precision and entity resolution change with scale.

3.2 LLM Model

The open-source LLM model I have chosen for both our Baseline and RAG systems is Ollama's `Mistral 7B`, a high-performance 7.3 billion parameter language model. I chose this model due to its recency and high performance given the smaller size. For my purposes, it was crucial to select a model that wasn't over-engineered for the desired task or too large for the average enthusiast to download for reproducing results.

3.3 RAG Retrieval Pipeline

HG-RAG operates in 4 stages when given a natural language query:

1. **Entity Resolution.** An LLM call scans the query for a named entity, which becomes the *anchor node* for traversal. If the LLM fails to extract a match, a fallback chain is used: exact match → first-word match → `difflib` fuzzy match (`cutoff=0.5`) → No entity found. If the fuzzy match returns a node of the wrong entity type (ex: a city when a country is expected), the system silently falls back to the known ground-truth anchor. This is particularly important in large-world settings where

Type	Example	Anchor
`local_fact`	"What does {city} export?"	city
`hierarchical`	"What country is {city} in?"	city
`neighborhood`	"Which cities trade with {city}?"	city
`multi_hop`	"A strike halts {exports} ... which cities are affected?"	city or country

Table 2: Query types used in testing.

name collisions are more likely, but also a limitation of the system worth mentioning.

2. **Hierarchy Anchoring (`k-up`).** After an anchor node is established, the pipeline traverses upwards through "contains" edges, collecting parent and grandparent nodes up to k steps. My default for k-steps upwards is 2, this captures the full three-level hierarchy.

3. **Lateral Traversal (`k-side`).** The process expands horizontally, collecting relational neighbors up to k steps along `borders`, `trade-with`, `hostile`, `unfriendly`, and `neutral` edges. To prevent adversarial context, which is the most important information for our queries, from being diluted by neutral neighbors when the subgraph cap is reached, hostile and unfriendly neighbors are always prioritized for inclusion.

4. **Child Traversal (`k_down`).** The pipeline can optionally collect child nodes downward through "contains" edges up to "k-down" steps. When the query requests such information, `k-down` increases from 0, which is the default.

The resulting subgraph is capped at 15 nodes to prevent context bloat. Retrieved nodes are serialized into a structured prompt block containing an explicit location chain (`"CityX is a city located in CountryY, on Planet Z"`), followed by labeled `[PLANET]`, `[COUNTRY]`, `[CITY]`, and `[RELATIONS]` blocks. In testing, arrow notion proved insufficient for LLM comprehension, prompting me to develop a different serialization.

3.4 Baseline

We compare HG-RAG against a dense vector RAG baseline that operates over the same knowledge graph without any structural awareness. At index time, each graph node (city, country, or planet) is serialized into a flat text chunk containing its attributes and direct relations, then encoded into a dense vector using `nomic-embed-text` (Nussbaum et al., 2024) via Ollama. At query time, the question is embedded with the same model and cosine similarity is used to retrieve the top-10 most similar chunks, which are concatenated and passed as context to the LLM. The value $k{=}10$ is chosen to match the approximate context size of HG-RAG's subgraph retrieval for a fair comparison. Unlike HG-RAG, the baseline has no knowledge of the hierarchy: it cannot walk up from a city to its country or planet, cannot prioritize structurally critical relationships, and relies entirely on embedding similarity to surface relevant nodes.

4 Evaluation Metrics

4.1 Query Design

We evaluate HG-RAG across four query types balanced at 25% each of the total 50 question pool per world size. Each query type tackles a specific challenge regarding traversing a hierarchical graph. Furthermore, all queries are deterministically generated from the graph at construction time, ensuring the benchmark is fully reproducible and that every query has a verifiable ground-truth answer.

`local_fact`: Local fact queries test whether the LLM can retrieve a single attribute directly from the anchor node. This is the most basic query type out of the 4, and does not require any context outside of the anchor.

`hierarchical`: Hierarchical queries require the system to traverse upward through the `contains` edge to identify a parent node.

`neighborhood`: Neighborhood queries test lateral local traversal, requiring the LLM to identify relational neighbors along `trade_with` or `borders` edges.

`multi_hop`: Multi-hop queries are the most demanding type, requiring the LLM to chain multiple relational steps: resolving an export, finding relevant trade partners, then identifying dependent cities.

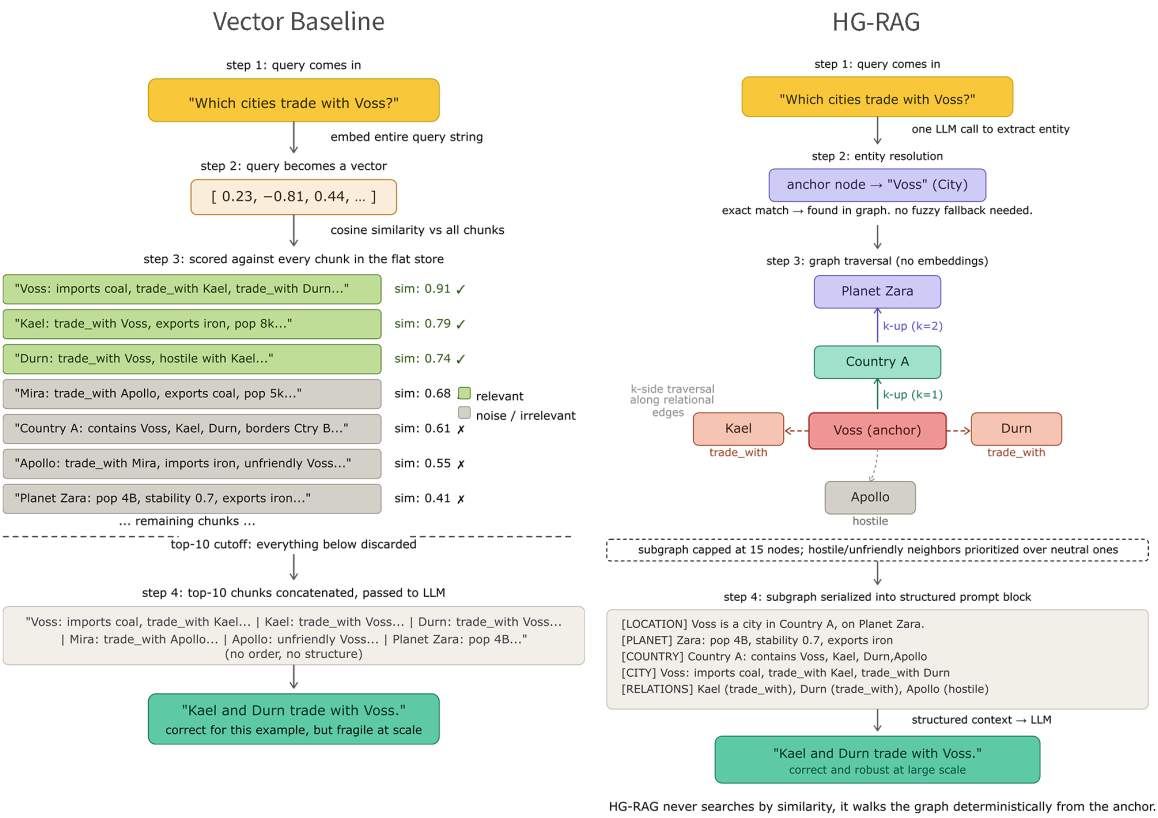

Figure 2: Side-by-side comparison of the vector baseline and HG-RAG retrieval pipelines for the query "Which cities trade with Voss?" The baseline embeds the query, ranks all chunks by cosine similarity, and passes the top-10 as flat context. HG-RAG resolves the named entity, traverses the hierarchy upward through `contains` edges, and expands laterally through relational edges, producing a structured prompt with explicit location chains and relevant labels.

To prevent incoherent queries on smaller worlds, multi-hop queries are only generated when their preconditions are satisfied: a trade disruption query requires the anchor's export to be imported by at least one trade partner, and a war query requires at least one hostile or unfriendly neighbor. Small worlds would regularly have scenarios where there were no suitable entities to fulfill the pre-made prompts for multi-hop queries, so if not possible the program would fall back on any of the three other types of queries. Each multi-hop query is accompanied by a deterministic answer key built directly from graph data at generation time, which is passed to the LLM judge during evaluation.

4.2 Rubric

I evaluate LLM performance through four metrics, all tackling different dimensions of quality:

1. **Factual accuracy** (`factual_accuracy`, 0-2) measures keyword overlap between the model's response and ground-truth answer. A full score of 2 is awarded when overlap reaches $\geq 70\%$, 1 for $\geq 30\%$, and 0 for anything below. This metric captures whether the model includes the correct entities and values from the retrieved context

2. **Hallucination Rate** (`hallucination_rate`, 0-1) measures the fraction of capitalized entity-like tokens in the response that do not appear anywhere in the knowledge graph. The lower the score, the better. This penalizes the model for inventing plausible-sounding entities that simply do not exist, a common failure when flat context overwhelms the LLM with loosely relevant information (Ji et al., 2023).

3. **Locality Awareness** (`locality_awareness`, 0-1) measures the fraction of graph nodes mentioned in the response that belong to the anchor's planet. This captures whether the model stays grounded in the relevant region of the hierarchy rather than drifting to unre-

World Size	Query Type	System	Factual Accuracy	Hallucination Rate	Locality Awareness
Small	Hierarchical	Baseline	0.50	0.000	1.00
		HG-RAG	2.00	0.000	1.00
	Local Fact	Baseline	1.00	0.021	1.00
		HG-RAG	2.00	0.042	1.00
	Multi-Hop	Baseline	1.18	0.053	0.88
		HG-RAG	1.42	0.030	1.00
	Neighborhood	Baseline	1.33	0.000	1.00
		HG-RAG	1.75	0.000	1.00
	Overall	Baseline	1.00	0.018	0.97
		HG-RAG	1.79	0.018	1.00
Medium	Hierarchical	Baseline	0.00	0.000	1.00
		HG-RAG	2.00	0.000	1.00
	Local Fact	Baseline	0.00	0.000	1.00
		HG-RAG	2.00	0.125	1.00
	Multi-Hop	Baseline	0.55	0.072	0.74
		HG-RAG	1.58	0.035	1.00
	Neighborhood	Baseline	0.42	0.008	0.97
		HG-RAG	1.58	0.000	1.00
	Overall	Baseline	0.24	0.020	0.93
		HG-RAG	1.79	0.040	1.00
Large	Hierarchical	Baseline	0.00	0.000	1.00
		HG-RAG	2.00	0.000	1.00
	Local Fact	Baseline	0.00	0.000	1.00
		HG-RAG	2.00	0.042	1.00
	Multi-Hop	Baseline	0.10	0.025	0.75
		HG-RAG	1.50	0.029	1.00
	Neighborhood	Baseline	0.00	0.000	1.00
		HG-RAG	1.87	0.000	1.00
	Overall	Baseline	0.02	0.006	0.94
		HG-RAG	1.86	0.017	1.00

Table 3: Average performance across 5 trials by query type and world size. Factual Accuracy (0–2), Hallucination Rate (0–1), Locality Awareness (0–1). Overall rows are bolded.

World Size	Query Type	System	Small	Medium	Large	Average
Multi-Hop	LLM Judge (1–5)	Baseline	3.45	2.82	1.66	2.64
		HG-RAG	3.23	3.88	4.10	3.74

Table 4: Multi-hop LLM judge scores (1–5) across world sizes.

lated parts of the world. I included this metric since it's a particularly important signal in large-world settings where many similar entities exist across different planets.

4. **LLM Judge Score** (`llm_judge_score`, 1-5) is utilized exclusively for multi-hop queries, where keyword overlap alone is insufficient to assess reasoning quality. Due to the open-ended nature of multi-hop questions, I created a special metric for them. The judge model receives the query, the model's response, and a deterministic answer key; then scores the response on a 1–5 rubric: 5 for correct entities with sound reasoning, 1 for wrong entities without reasoning. The rubric deliberately weights entity identification over reasoning quality, since LLMs are known to fabricate plausible reasoning chains even when their factual grounding is incorrect.

One limitation worth noting: I use the same `Mistral 7B` instance as both the answering model and the LLM judge. While this is a practical constraint, it introduces the possibility of self-consistency bias (Zheng et al., 2023): the judge may be more lenient toward responses that mirror its own reasoning patterns. I treat `llm_judge_score` as a supplementary signal alongside the deterministic metrics rather than a primary measure of correctness.

5 Experiment and Results

Five trials were conducted per world size (small, medium, large), with 50 deterministically generated queries per trial split equally across the four query types. All experiments were run on a `2024 ASUS ROG Zephyrus G14` (`AMD Ryzen 9 8945HS`, `NVIDIA RTX 4060 Laptop GPU`, `16GB RAM`), with `Mistral 7B` operated locally via `Ollama` (Ollama Team, 2023) at `Q4_K_M`. Runtime for one trial was

approximately 5 minutes.

5.1 Baseline Collapse at Scale

The embedding-based RAG baseline performs reasonably on small worlds, achieving an overall factual accuracy of 1.004. However, this performance rapidly deteriorates with scale: factual accuracy falls to 0.242 on medium worlds and to 0.022 on large worlds. Despite using semantic retrieval to surface relevant chunks, the baseline scores 0.00 on hierarchical and local fact queries at both medium and large scales. This suggests that cosine similarity over flat node descriptions fails to consistently retrieve structurally relevant nodes; for example, the parent country for a city may not be semantically close to the query, even though it is the correct answer. The baseline's strongest results occur on multi-hop queries, where it achieves 1.184 on small worlds, but even here performance degrades to 0.100 at large scale.

One notable pattern in the baseline's failure mode is that hallucination rates remain low even as factual accuracy collapses. At large scale, the baseline achieves a hallucination rate of just 0.006 while scoring near-zero on accuracy. This suggests that when semantic retrieval surfaces the wrong nodes, the LLM faithfully reports their contents rather than inventing entities, producing answers that are internally consistent but factually incorrect. This suggests that my hallucination rate metric alone is not a reliable proxy for answer quality.

5.2 HG-RAG Performance

HG-RAG maintains strong performance across all world sizes. Overall factual accuracy is 1.792 for small worlds, 1.792 for medium, and 1.857 for large. On hierarchical and local fact queries, HG-RAG achieves a perfect 2.00 at every scale, confirming that the traversal pipeline reliably surfaces the anchor node and its structural parents. Neighborhood queries show consistent gains as well, rising from 1.750 at small scale to 1.867 at large scale.

Locality awareness remains at a perfect 1.00 across all world sizes and query types for HG-RAG, meaning the system never drifts to entities outside the anchor's planet. The baseline, by contrast, shows locality degradation on multi-hop queries in particular, dropping to 0.75 at large scale. This confirms that hierarchical anchoring keeps the LLM grounded in the relevant region of the graph.

Hallucination rates for HG-RAG are generally low, averaging 0.018 on small worlds, 0.040 on medium, and 0.017 on large. One outlier is the local fact hallucination rate of 0.125 on medium worlds, which is higher than any other HG-RAG condition. This warrants further investigation through further research, but does not undermine the broader trend.

5.3 Multi-Hop Scaling

Multi-hop queries have proved to be the most demanding type for both systems, requiring the LLM to chain multiple relational steps to arrive at an answer. The LLM judge scores reveal an inverse scaling pattern between the two systems. The baseline degrades as world size increases ($3.450 \rightarrow 2.817 \rightarrow 1.660$), consistent with its broader collapse under increased noise. HG-RAG, by contrast, improves with scale ($3.233 \rightarrow 3.883 \rightarrow 4.100$), achieving its strongest multi-hop performance on the largest and most complex world.

5.4 Why Small Worlds Narrow the Gap

An initially counterintuitive finding is that HG-RAG does not perform notably better on small worlds than on medium or large ones. You may expect a smaller graph would make retrieval easier due to less noise. Two factors contribute to this. First, with only $\sim$18 nodes, the 15-node subgraph cap means HG-RAG retrieves nearly the entire graph, eroding the signal-to-noise advantage that hierarchical traversal provides at larger scales. The retrieved context, despite being structured and locally anchored, contains a high proportion of nodes only loosely relevant to the query. Second, the baseline itself faces less noise in small worlds. With fewer entities competing for attention, semantic retrieval is less likely to surface irrelevant chunks. In larger worlds, both effects reverse: the subgraph cap acts as a meaningful filter that isolates a tight neighborhood from a vast graph, while the baseline's retrieval struggles to identify structurally relevant nodes among hundreds of candidates. This explains why HG-RAG's strongest results occur at large scale rather than small scale: scale amplifies the benefit of structured retrieval rather than undermining it.

6 Conclusion

This paper presented HG-RAG, a retrieval-augmented generation framework that applies graph-traversal heuristics over a hierarchical knowledge graph to deliver structured and relevant context to a language model. When evaluated across

three world scales and four query types, HG-RAG consistently outperformed an embedding-based RAG baseline that retrieves context via cosine similarity over dense node vectors. While the baseline achieves reasonable performance on small worlds, its factual accuracy collapses to 0.022 at large scale, even with semantic retrieval actively selecting the top-k most relevant chunks. Conversely, HG-RAG maintains an overall factual accuracy of 1.857 on the same large worlds. This gap demonstrates that semantic similarity alone is insufficient when the knowledge structure is hierarchical: a system must understand not just what is relevant, but where it sits in the hierarchy.

The scaling behavior of multi-hop queries reinforces this conclusion. While multi-hop proves to be the hardest query type for both systems, the trend tells two very different stories. The baseline's LLM judge score degrades as the world grows ($3.450 \rightarrow 2.817 \rightarrow 1.660$), consistent with its broader inability to surface structurally relevant nodes at scale. On the other hand, HG-RAG improves with scale ($3.233 \rightarrow 3.883 \rightarrow 4.100$). Its best multi-hop performance occurs on the largest, most complex world. In small worlds, the traversal advantage is partially offset by context noise from an oversaturated subgraph, and by the baseline facing less retrieval pressure with fewer competing entities. In large worlds, the retrieval is doing exactly what it was designed to do: rounding up a tight, relevant neighborhood from a vast graph. These results suggest that as knowledge graphs grow in complexity, the case for hierarchy-aware retrieval over semantic retrieval only strengthens.

Limitations and Further Improvements

The subgraph cap is currently fixed at 15 nodes regardless of world size. In future work, scaling this parameter proportionally with graph size may improve the lackluster performance on small worlds, where the cap captures nearly the entire graph, while preserving the filtering benefit at larger scales.

A significant methodological concern is the evaluation pipeline for open-ended responses. Both the answering model and the grader are the same `Mistral 7B` instance, which as noted earlier, could possess a self-consistency bias. Compounding this, LLM-based grading carries an inherent hallucination risk independent of model consistency. The grader may reward responses that sound correct rather than being correct. This was observed during testing: the judge awarded a score of 5 to two incorrect responses, likely because they were structured as detailed bullet points and appeared substantive despite being factually wrong. This highlights a fundamental weakness of LLM-as-judge evaluation and suggests that future iterations should use a stronger, separate judge model.

Another pitfall of HG-RAG is the inability to handle queries that do not contain a named entity. To recap, the framework's first step is to perform one LLM pass to locate a named entity. Without a named-entity of any kind (`city`, `country`, `planet`), there is no way for the system to answer the question. Therefore, an attribute-based query such as "`Which city has 2 thousand people and exports coal?`" that can be answered using the map, fall outside the current system's capabilities. However, this is not a fundamental flaw in the framework's design. A reverse-lookup or attribute-filtering subroutine could be added to handle these cases. The decision to omit it was deliberate, as the added complexity would have obscured the core contribution of hierarchy-guided retrieval.

References

Nilesh Chakraborty, Denis Lukovnikov, Gaurav Maheshwari, Priyansh Trivedi, Jens Lehmann, and Asja Fischer. 2021. Introduction to neural network based approaches for question answering over knowledge graphs. *arXiv preprint arXiv:1907.09361*.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Aric Hagberg, Pieter Swart, and Daniel Chult. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference*.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Ethel Zhu, Sejin Lee, Fei Kong, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Jiawei Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.

Yunshi Lan and Jing Jiang. 2020. Query graph generation for answering multi-hop complex questions from knowledge bases. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 969–974, Online. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. 2024. Nomic embed: Training a reproducible long context text embedder. *Preprint*, arXiv:2402.01613.

Ollama Team. 2023. Ollama. https://ollama.com.

Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval augmentation reduces hallucination in conversation. *Preprint*, arXiv:2104.07567.

Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 535–546, Online. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*.